

Module 4

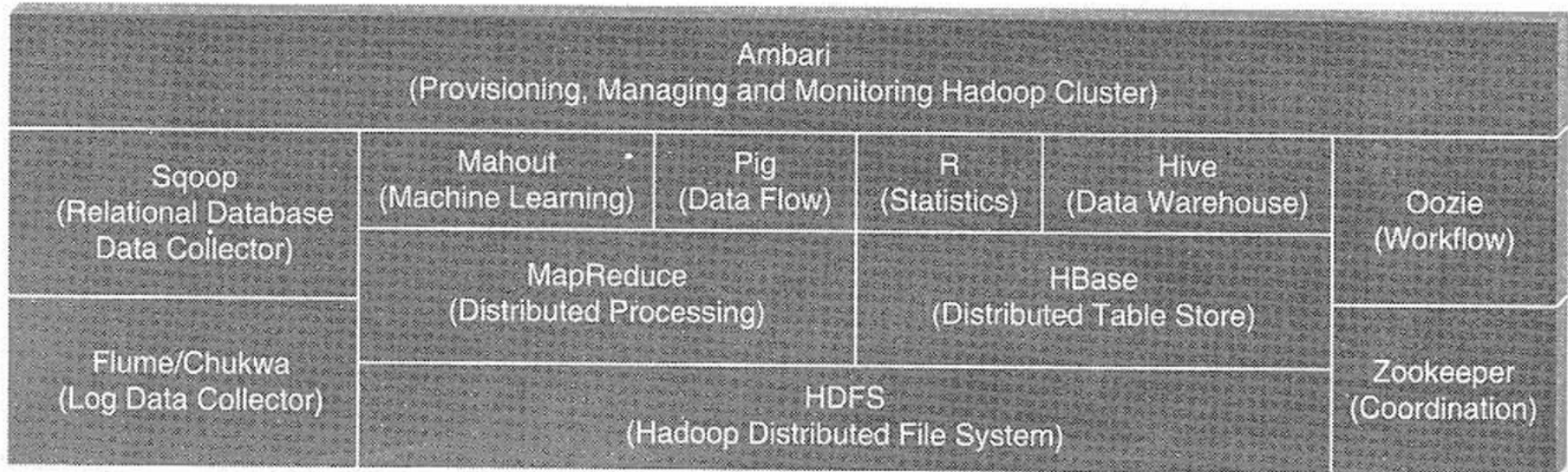
Introduction to Hive: What is Hive, Hive Architecture, Hive data types, Hive file formats, Hive Query Language (HQL), RC File implementation, User Defined Function (UDF).

Introduction to Pig: What is Pig, Anatomy of Pig, Pig on Hadoop, Pig Philosophy, Use case for Pig, Pig Latin Overview, Data types in Pig, Running Pig, Execution Modes of Pig, HDFS Commands, Relational Operators, Eval Function, Complex Data Types, Piggy Bank, User Defined Function, Pig Vs Hive.

Chapter 1

Hive

- **Data Warehousing** is the **process of collecting, storing, and managing large volumes of data from different sources** in a **central repository**, called a **data warehouse**, for the **purpose of analysis, reporting, and decision-making**.



Example

Let's say there are **5 supermarket branches**. Each one has its own system to record:

- What was sold
- When it was sold
- Who bought it

All this data is sent to a **central warehouse** (a data warehouse). There, the data is:

- **Collected** from each store
- **Cleaned** (so formats match, errors are removed)
- **Organized** into categories like "Products", "Sales", "Customers", "Time", etc.

Now the manager can:

- Check total sales from all branches in one place
- Compare sales between two months
- Predict future sales based on past trends

What is Hive?

- **Hive** is a **data warehousing tool** built on top of **Hadoop**. It helps to **store, process, and analyze large structured data** using a language similar to SQL, called **HiveQL**.

What Hive does?

- **Summarization** – Combine or summarize data (e.g., total sales per month).
- **Querying** – Ask questions like in SQL (e.g., "How many customers bought item X?").
- **Analysis** – Study patterns or trends (e.g., customer buying behavior).

- **Facebook** initially **created Hive component** to **manage their** ever-growing **volumes of log data**. Later **apache software foundation developed** it as **a open-source** and it came to be known as **apache Hive**.
- Hive makes use of the following:
 - **HDFS** – To store the data (Hadoop's storage system).
 - **MapReduce** – To run the processing jobs (splits tasks and runs them in parallel).
 - **RDBMS** – To store **metadata** (like table names, column names, data types, etc).
- Hive provides **HQL (Hive Query Language)** or HiveQL which is similar to SQL.
- It is designed to **support OLAP** (Online Analytical Processing).
- Hive provides **extensive data type functions** and **formats** for **data summarization** and **analysis**.

OLAP stands for Online Analytical Processing.

It is a technology used to **analyze large amounts of data** and get **useful insights and reports** from it.

What is OLTP

OLTP systems are used for **day-to-day operations** where many users perform **quick, real-time transaction** like:

- Inserting data
- Updating data
- Deleting data
- Retrieving a single record quickly

Feature	OLAP	OLTP
Purpose	Reporting and Analysis	Day-to-day transactions
Data type	Historical, large volumes	Real-time, current data
Operations	Read-heavy (queries)	Write-heavy (insert/update)
Examples	Business reports, trends	Banking systems, e-commerce

History of Hive

- 2007:

Hive was **created at Facebook** to handle their growing log data.

(Logs = records of user actions like clicks, likes, posts.)

- 2008:

Hive became an **Apache open-source project** and part of the Hadoop ecosystem.

(This means anyone could use or contribute to it.)

Recent Releases of Hive

Hive 0.10	Hive 0.13	Hive 0.14
• Used for Batch processing (processes large data in chunks). • Supported Read-only data (no updates, just analysis). • Used HiveQL (SQL-like language). • Used MapReduce (MR) for running jobs.	• Added Interactive query support (faster and more flexible). • Still Read-only, but supported more advanced SQL features. • Could run using MapReduce or TEZ (TEZ is faster than MR).	• Introduced ACID transactions (like a real database: support for insert, update, delete). • Added a Cost-based optimizer (chooses the best way to run queries). • Supported SQL temporary tables (used for temporary calculations).

Hive features

- Hive uses a language called **HQL (Hive Query Language)**, which looks a lot like SQL.
- **Writing HQL** is **simple and readable**, like writing plain English queries.
- Hive supports **rich data types** such as **structs, lists and maps**.
- Hive supports **SQL filters, group-by and order-by** clauses.
- **Custom types, custom functions** can be defined (You can create your own **data types** and **functions** in Hive.)

Hive Integration and workflow

- Hourly Log Data can be stored directly into HDFS and then data cleansing is performed on the log file.
- Finally, Hive table(s) can be created to query the log file.

Note:

- Log file - **Log files** are files that **record events and activities** happening in a system, app, or website.
- They help in **tracking user actions, errors, and system behavior** for **monitoring, debugging, and analysis**.

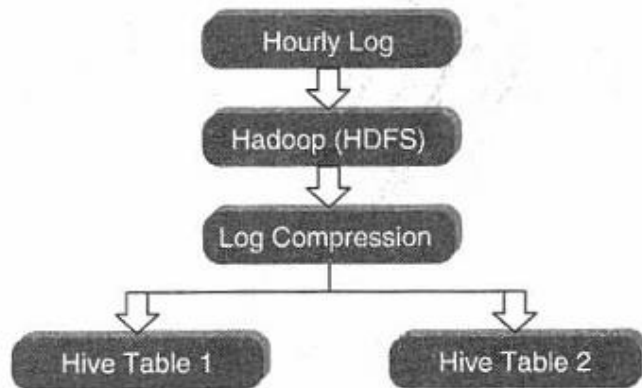


Figure 9.4 Flow of log analysis file.

Hive data units

1. **Databases:** Think of this as a **container or folder** that **holds different tables**. It's where **all the tables in Hive live**.
2. **Tables:** A table is like a spreadsheet, where you **store records** (rows of data) **that follow the same structure (schema)**. **Each column** in the **table holds** a **specific type** of **data**.
3. **Partitions:** Imagine **dividing a large table** into **smaller chunks** based on **certain criteria**, like **dates or regions**. Hive **creates separate folders** to **store data** according to these **categories**. For example, if you're tracking sales by region, Hive might create separate partitions for each region.
4. **Buckets (or Clusters):** Buckets are another way to organize data, but instead of using categories like regions, **Hive uses a "hash" function** to **distribute the data evenly** into **a set number of "buckets."** This helps in **efficiently managing and processing data**.
 1. You choose a column (like `user_id`, `customer_id`, or `employee_id`) as the bucket column.
 2. Hive applies a **hash function** to the value in that column.
 3. Then it calculates:

```
bucket_number = hash(column_value) % number_of_buckets
```
 4. Based on the result, the record is placed into one of the buckets.

When to use partitioning/bucketing?

- **Bucketing works well** when the **field has high cardinality** (cardinality is the number of values a column or field can have) and **data is evenly distributed among buckets**.

Cardinality means the number of **unique values** in a column.

- Example: A "Gender" column has **low cardinality** (only 'Male', 'Female').
 - A "User ID" column has **high cardinality** (many unique values).
- **Partitioning works best** when the **cardinality of the partitioning field is not too high**. **Partitioning can be done on multiple fields** with an order (Year/Month/Day) whereas **bucketing can be done on only one field**.

Imagine we have a large **sales table** that stores millions of sales records from different regions and over multiple years.

To optimize performance, we can **partition** the table by `year` and `region`.

Hive Storage Structure

- A **Database** → contains multiple **Tables**
- A **Table** → contains **rows and columns**
- In Hive:
 - A **table** is stored as a **folder**.
 - If the table is **partitioned**, each partition is a **sub-folder**.
 - If the table is **bucketed**, each bucket is a **file** inside those folders.

/sales_db/	← Database folder
/sales/	← Table folder
/region=North/	← Partition folder (based on 'region')
part-0000	← Bucket file 1
part-0001	← Bucket file 2
...	

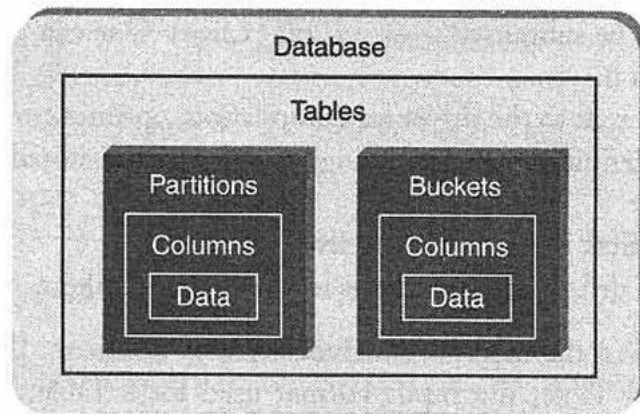


Figure 9.5 Data units as arranged in a Hive.

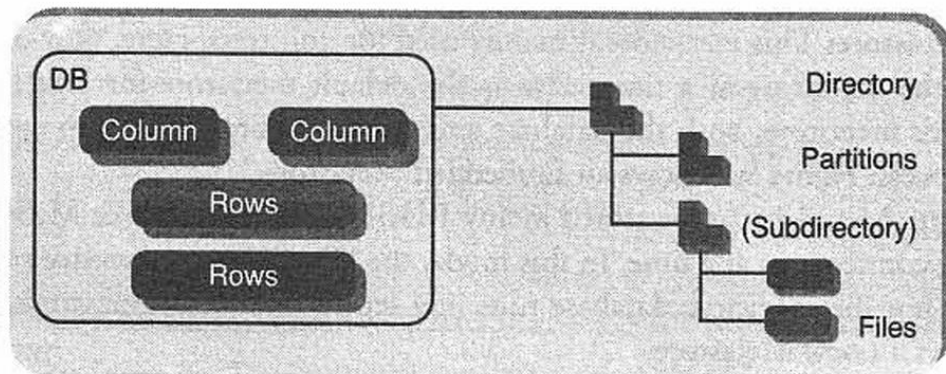


Figure 9.6 Semblance of Hive structure with database.

Hive Architecture

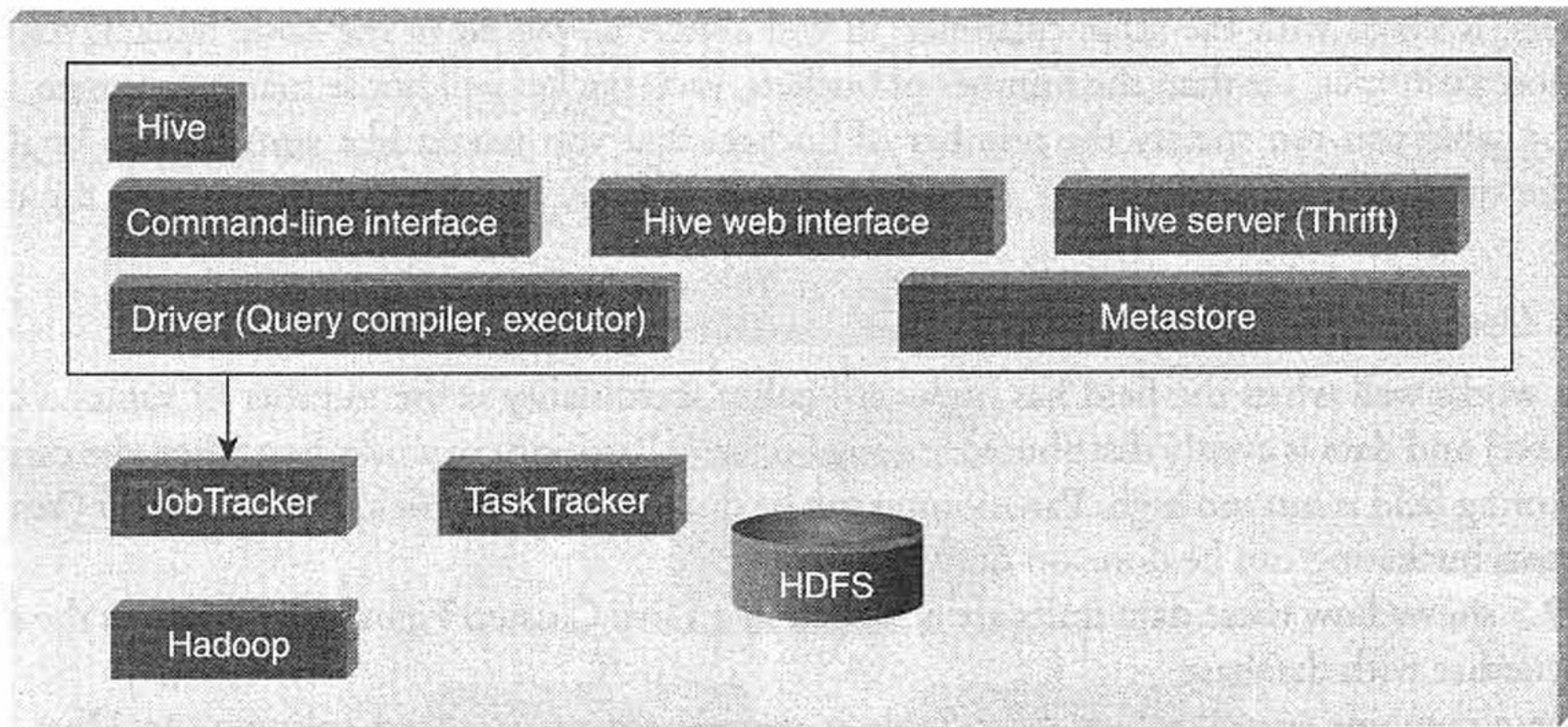


Figure 9.7 Hive architecture.

Hive CLI (Command Line Interface)

- This is a text-based tool where you **can type Hive queries**.
- Most commonly **used by developers and data engineers**.

Hive Web Interface

- A Graphical User Interface (GUI) where you **can write and run Hive queries using a browser**.
- **Good for beginners** or those who prefer not to use the terminal.

Hive Server

An optional server that **allows remote clients (users on other systems) to connect and submit Hive jobs**.

JDBC/ODBC Clients

You can also **write programs** (like in Java or Python) that **connect to Hive** and **submit queries using JDBC or ODBC drivers**.
(JDBC – Java Database Connectivity (Used specifically for Java)
ODBC - Open Database Connectivity (used with C, C++, Python, etc.))

Driver

This is the **brain of Hive**. When you submit a query:

- The **Driver receives** it,
- it **compiles the query**,
- **Optimizes it**,
- And **sends it to execution engines** like MapReduce, Tez, or Spark.

Metastore

The **Metastore** stores all the information about:

- **Tables, Columns, Partitions**
- **where the data is stored**
- **formats used (text, ORC, etc.)**

Every time you **create, update or delete a table**, the **Metastore** is updated.

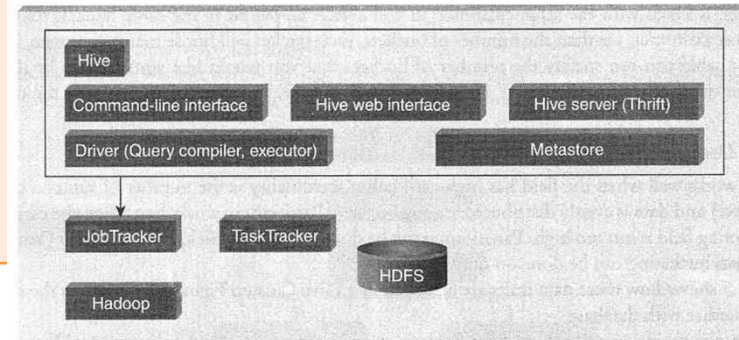


Figure 9.7 Hive architecture.

Types of Metastore

1. Embedded Metastore

- For **testing** or **small use**
- Uses Apache Derby DB
- Only **one user** can access at a time
- Everything runs in a **single process**

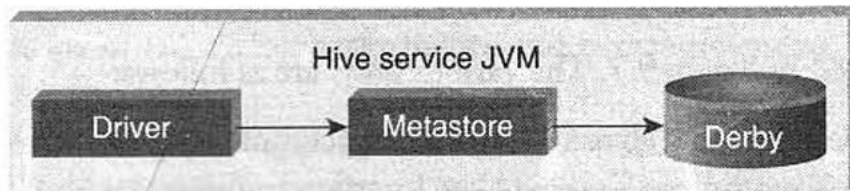


Figure 9.8 Embedded Metastore.

2. Local Metastore

- Used for **multi-user access**
- Metadata is stored in an **external database** (like MySQL)
- Hive and the database can be on the **same or different machines**

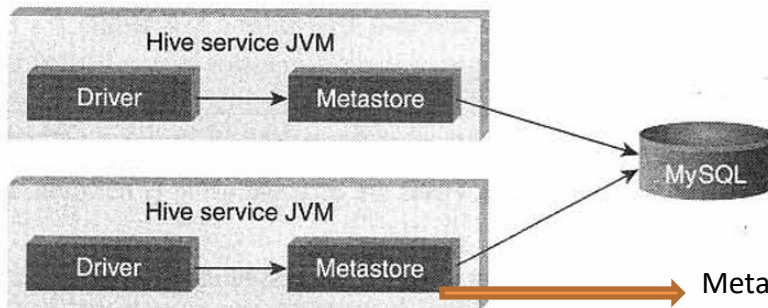


Figure 9.9 Local Metastore.

Metastore manages metadata stored in external database

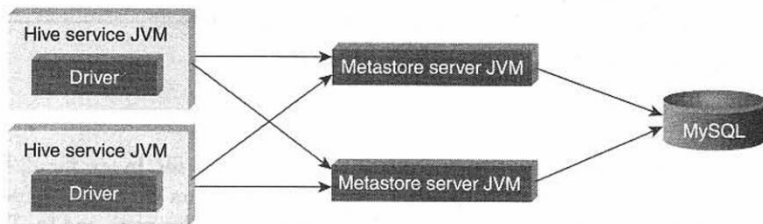


Figure 9.10 Remote Metastore.

3. Remote Metastore

- Used in **large-scale enterprise** setups
- Hive and the Metastore run on **different machines**
- Offers better **security** and **scalability**

Type	Suitable For	Storage DB	Access	Setup Location
Embedded	Testing / Single user	Derby	One user	Same process
Local	Multi-user	MySQL (external)	Multiple users	Same or different machine
Remote	Enterprise setup	MySQL (external)	Multiple users	Different machines

Hive Data types

Primitive Data Types

Numeric Data Type

TINYINT	1-byte signed integer
SMALLINT	2-byte signed integer
INT	4-byte signed integer
BIGINT	8-byte signed integer
FLOAT	4-byte single-precision floating-point
DOUBLE	8-byte double-precision floating-point number

String Types

STRING	
VARCHAR	Only available starting with Hive 0.12.0
CHAR	Only available starting with Hive 0.13.0

Strings can be expressed in either single quotes (') or double quotes (")

Miscellaneous Types

BOOLEAN	
BINARY	Only available starting with Hive

Collection Data Types

Collection Data Types

- | | |
|--------|---|
| STRUCT | Similar to 'C' struct. Fields are accessed using dot notation. E.g.: struct('John', 'Doe') |
| MAP | A collection of key-value pairs. Fields are accessed using [] notation. E.g.: map('first', 'John', 'last', 'Doe') |
| ARRAY | Ordered sequence of same types. Fields are accessed using array index. E.g.: array('John', 'Doe') |
-

Hive File Format

Feature	Text File	Sequential File	RCFile (Record Columnar File)
Readability	Human-readable (e.g., CSV)	Not human-readable (binary)	Not human-readable (column-wise binary)
Format	Line-based, uses delimiters	Key-value pairs in binary	Column-wise inside row groups
Delimiters Used	Yes (<code>^A</code> , <code>^B</code> , <code>^C</code> , <code>\n</code>)	No (binary format)	No (columnar binary format)
Compression Support	No (unless manually applied)	Yes	Yes
Efficiency	Least efficient	Medium	Most efficient for analytical queries
Use Case	Simple data import/export	Faster processing with compression	Fast queries on large data, especially columns
Examples	CSV, TSV, JSON, XML	Binary key-value pairs	Column-based storage (split rows + columns)

1. Text File Format (CSV/TSV)

- Each line = one record.
- Fields are separated by commas.
- You can also use tab (TSV) or custom delimiters like ^A.
- Special characters are used to **separate fields**:

^A separates fields (like columns).

^B separates items in arrays or structures.

^C separates key-value pairs.

\n is for a new line (end of a record).

101,John,25,New York

102,Alice,30,Los Angeles

2. Sequential File

- These are **binary files** (not human-readable).
- Data is stored as **key-value pairs**.
- These files **support compression**

BinaryKey: 101

BinaryValue: John,25

3. RCFile (Record Columnar File)

- This stores data in a **column-based format**.
- It helps in **fast data processing**, especially for **aggregation operations** (like SUM, AVG).
- It splits the data:
 - **Horizontally** – into row groups.
 - **Vertically** – each column's values are stored together (column-wise).
- This format is **more efficient** than row-wise storage, especially for analytics.

Table 9.1 A table with four columns

C1	C2	C3	C4
11	12	13	14
21	22	23	24
31	32	33	34
41	42	43	44
51	52	53	54

Table 9.2 Table with two row groups

Row Group 1				Row Group 2			
C1	C2	C3	C4	C1	C2	C3	C4
11	12	13	14	41	42	43	44
21	22	23	24	51	52	53	54
31	32	33	34				

Table 9.3 Table in RCFile Format

Row Group 1	Row Group 2
11, 21, 31;	41, 51;
12, 22, 32;	42, 52;
13, 23, 33;	43, 53;
14, 24, 34;	44, 54;

HiveQL - Database

- A database is like a container for data. It has a collection of tables which houses the data.

Objective: To create a database named “STUDENTS” with comments and database properties.

Act:

```
CREATE DATABASE IF NOT EXISTS STUDENTS COMMENT 'STUDENT Details'  
WITH DBPROPERTIES ('creator' = 'JOHN');
```

Outcome:

```
hive> CREATE DATABASE IF NOT EXISTS STUDENTS COMMENT 'STUDENT Details' WITH DBPROPERTIES ('creator'  
r' = 'JOHN');  
OK  
Time taken: 0.536 seconds  
hive>
```

IF NOT EXIST: It is an optional clause. The create database statement with “IF Not EXISTS” clause creates a database if it does not exist. However, if the database already exists then it will notify the user that a database with the same name already exists and will not show any error message.

COMMENT: This is to provide short description about the database.

WITH DBPROPERTIES: It is an optional clause. It is used to specify any properties of database in the form of (key, value) separated pairs. In the above example, “Creator” is the “Key” and “JOHN” is the value.

We can use “SCHEMA” in place of “DATABASE” in this command.

Note: We have not specified the location where the Hive database will be created. By default all the Hive databases will be created under default warehouse directory (set by the property `hive.metastore.warehouse.dir`) as `/user/hive/warehouse/database_name.db`. But if we want to specify our own location, then the `LOCATION` clause can be specified. This clause is optional.

Objective: To display a list of all databases.

Act:

```
SHOW DATABASES;
```

Outcome:

```
hive> SHOW DATABASES;  
OK
```

```
students  
Time taken: 0.082 seconds, Fetched: 22 row(s)  
hive>
```

By default, **SHOW DATABASES** lists all the databases available in the metastore. We can use “**SCHEMAS**” in place of “**DATABASES**” in this command. The command has an optional “Like” clause. It can be used to filter the database names using regular expressions such as “*”, “?”, etc.

SHOW DATABASES LIKE “Stu*”

SHOW DATABASES like “Stud???s”

* -> Any number of characters.

? -> exactly one character.

Objective: To describe a database.

Act:

DESCRIBE DATABASE STUDENTS;

Note: Shows only DB name, comment, and DB directory.

Outcome:

```
hive> DESCRIBE DATABASE STUDENTS;
OK
students      STUDENT Details hdfs://volgalnx010.ad.infosys.com:9000/user/hive/warehouse/studen
ts.db         root      USER
Time taken: 0.03 seconds, Fetched: 1 row(s)
hive>
```

Objective: To describe the extended database.

Act:

DESCRIBE DATABASE EXTENDED STUDENTS;

Note: Shows DB properties also.

Outcome:

```
hive> DESCRIBE DATABASE EXTENDED STUDENTS;
OK
students      STUDENT Details hdfs://volgalnx010.ad.infosys.com:9000/user/hive/warehouse/studen
ts.db         root      USER      {creator=JOHN}
Time taken: 0.027 seconds, Fetched: 1 row(s)
hive>
```

DESCRIBE DATABASE EXTENDED shows database's properties given under DBPROPERTIES argument at the time of creation.

We can use "SCHEMA" in place of "DATABASE", "DESC" in place of "DESCRIBE" in this command.

Objective: To alter the database properties.

Act:

```
ALTER DATABASE STUDENTS SET DBPROPERTIES ('edited-by' = 'JAMES');
```

Note: We can use the “ALTER DATABASE” command to

- Assign any new (key, value) pairs into DBPROPERTIES.
- Set owner user or role to the Database.

In Hive, it is not possible to unset the DB properties.

Outcome:

```
hive> ALTER DATABASE STUDENTS SET DBPROPERTIES ('edited-by' = 'JAMES');  
OK  
Time taken: 0.086 seconds  
hive> █
```

In the above example, the ALTER DATABASE command is used to assign new ('edited-by' = 'JAMES') pair into DBPROPERTIES. This can be verified by using the 'describe *extended*'.

```
Hive> DESCRIBE DATABASE Student EXTENDED
```

Objective: To drop database.

Act:

DROP DATABASE STUDENTS;

Note: Hive creates database in the warehouse directory of Hive as shown below:

Contents of directory `/user/hive/warehouse`

Goto:

Go to parent directory

Name	Type	Size	Replication	Block Size	Modification Time	Permission	Owner	Group
students.db	dir				2015-02-24 21:50	rw-r--r--	root	supergroup

Now assume that the database "STUDENTS" has 10 tables within it. How do we delete the complete database along with the tables contained therein?

Use the command:

DROP DATABASE STUDENTS CASCADE;

By default the mode is RESTRICT which implies that the database will NOT be dropped if it contains tables.

Note: The complete syntax is as follows:

```
DROP DATABASE STUDENTS;
```

- This command is used to **delete the database** named `STUDENTS`.
- But it works **only if the database is empty** (i.e., no tables inside).
- If there are tables, Hive will throw an error.

Where is the database stored?

Hive stores databases as folders in its **warehouse directory** (usually at `/user/hive/warehouse`).

`/user/hive/warehouse/students.db`

If the **STUDENTS** database has tables (e.g., 10 tables), the basic **DROP DATABASE** command will **not work**.

```
DROP DATABASE STUDENTS CASCADE;
```

- `CASCADE` means: **Drop the database AND all its tables**.
- This will delete the database **even if it contains tables or data**.

If the **CASCADE** is **not used**, then by default, it will be in **RESTRICT mode**. It means, `DROP DATABASE STUDENTS;` will fail if `STUDENTS` contains any tables.

Tables

Hive has **2 types of tables**:

- 1. Managed Table (also called Internal Table)**
- 2. External Table**

1. Managed Table

- This is the **default type** of table in Hive.
- The data is **not used by any other system** outside Hive.

Features:

- **Storage Location:**
 - Stored inside Hive's **warehouse directory** (/user/hive/warehouse).
- **Fully Managed by Hive:**
 - Hive **controls both the data and the table**.
 - You don't have to manage the file location manually.
- **Drop Behavior:**
 - If you **drop** (delete) the table:
 - Hive **deletes both** the table structure (metadata) **and** the data (files on disk).

- When you create a **table** in Hive, by **default** it is **internal or managed table**. If one needs to create an external table, one will have to use the keyword **“External”**.

Objective: To create managed table named 'STUDENT'.

Act:

```
CREATE TABLE IF NOT EXISTS STUDENT(rollno INT,name STRING,gpa FLOAT) ROW  
FORMAT DELIMITED FIELDS TERMINATED BY '\t';
```

ROW FORMAT DELIMITED

- Tells Hive how to read each **row of data**.

FIELDS TERMINATED BY '\t'

- Specifies that **fields/columns in each row** are separated by a **tab character (\t)**.

```
101  John  8.5
```

Outcome:

```
hive> CREATE TABLE IF NOT EXISTS STUDENT(rollno INT,name STRING,gpa FLOAT) ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t';
OK
Time taken: 0.355 seconds
hive>
```

Objective: To describe the “STUDENT” table.

Act:

DESCRIBE STUDENT;

Outcome:

```
hive> DESCRIBE STUDENT;
OK
rollno          int
name            string
gpa             float
Time taken: 0.163 seconds, Fetched: 3 row(s)
hive>
```

- Hive creates managed table in the warehouse directory of Hive as shown below:
/user/hive/warehouse/students.db
- To check whether an existing table is managed or external:

```
DESCRIBE FORMATTED tablename;
```

DESCRIBE	This keyword is used in Hive to show metadata (information) about a table.
FORMATTED	This keyword tells Hive to give a detailed and nicely formatted output including file location, table type, owner, storage info , etc.
tablename	Replace this with your actual table name. Example: STUDENT

It displays complete metadata of the specified table. Among that, look for

Table Type: MANAGED_TABLE

Table Type: EXTERNAL_TABLE

Managed table	Hive manages both data and metadata .
External table	Hive manages only metadata ; data is outside Hive's control .

2. External or self managed table

1. When the table is dropped, it retains the data in the underlying location.
2. **External** keyword is used to create an external table.
3. **Location** needs to be specified to store the dataset in that particular location.

Objective: To create external table named 'EXT_STUDENT'.

Act:

```
CREATE EXTERNAL TABLE IF NOT EXISTS EXT_STUDENT(rollno INT,name  
STRING,gpa FLOAT) ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t'  
LOCATION '/STUDENT_INFO';
```

Outcome:

```
hive> CREATE EXTERNAL TABLE IF NOT EXISTS EXT_STUDENT(rollno INT,name STRING,gpa FLOAT) ROW FORM  
T DELIMITED FIELDS TERMINATED BY '\t' LOCATION '/STUDENT_INFO';  
OK  
Time taken: 0.123 seconds  
hive>
```

Note: Hive creates the external table in the specified location.

CREATE EXTERNAL TABLE	Tells Hive to create a table that references external data . Hive will not delete the data when the table is dropped.
IF NOT EXISTS	Prevents an error if the table already exists — Hive won't create a duplicate.
EXT_STUDENT	The name of the table being created. "EXT" is a naming convention to indicate it is an external table.
(rollno INT, name STRING, gpa FLOAT)	The table schema with three columns: - rollno: Integer type - name: String type - gpa: Float (decimal) number
ROW FORMAT DELIMITED	Specifies the format of the input file is delimited (i.e., values are separated by some character)
FIELDS TERMINATED BY '\t'	Specifies that each field in a row is separated by a tab character (\t)
LOCATION '/STUDENT_INFO'	Tells Hive where to find the actual data files on HDFS . Hive will not move or delete the files at this location.

Concept	Managed Table	External Table
Like a locker	You put your stuff in Hive's locker, and Hive manages it	You tell Hive where your locker is, but Hive doesn't own it

Loading data into table from file

Objective: To load data into the table from file named student.tsv.

Act:

```
LOAD DATA LOCAL INPATH '/root/hivedemos/student.tsv' OVERWRITE INTO TABLE EXT_STUDENT;
```

Note: Local keyword is used to load the data from the local file system. In this case, the file is copied. To load the data from HDFS, remove local key word from the statement. In this case, the file is moved from the original location.

Outcome:

```
hive> LOAD DATA LOCAL INPATH '/root/hivedemos/student.tsv' OVERWRITE INTO TABLE EXT_STUDENT;
Loading data to table students.ext_student
Table students.ext_student stats: [numFiles=0, numRows=0, totalSize=0, rawDataSize=0]
OK
Time taken: 5.034 seconds
hive>
```

Let us understand the difference between INTO TABLE and OVERWRITE TABLE with an example:

Assume the “EXT_STUDENT” table already had 100 records and the “student.tsv” file has 10 records. After issuing the LOAD DATA statement with the INTO TABLE clause, the table “EXT_STUDENT” will contain 110 records; however, the same LOAD DATA statement with the OVERWRITE clause will wipe out all the former content from the table and then load the 10 records from the data file.

9.5.5.4 Collection Data Types

Objective: To work with collection data types.

Input:

1001,John,Smith:Jones,Mark1!45:Mark2!46:Mark3!43

1002,Jack,Smith:Jones,Mark1!46:Mark2!47:Mark3!42

Act:

```
CREATE TABLE STUDENT_INFO (rollno INT,name String, sub ARRAY<STRING>,marks
MAP<STRING,INT>)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
COLLECTION ITEMS TERMINATED BY ':'
MAP KEYS TERMINATED BY '!';
LOAD DATA LOCAL INPATH '/root/hivedemos/studentinfo.csv' INTO TABLE
STUDENT_INFO;
```

Outcome:

```
hive> CREATE TABLE STUDENT_INFO (rollno INT,name String, sub ARRAY<STRING>,marks MAP<STRING,INT>)
> ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
> COLLECTION ITEMS TERMINATED BY ':'
> MAP KEYS TERMINATED BY '!';
OK
Time taken: 0.112 seconds
hive>
```

```
hive> LOAD DATA LOCAL INPATH '/root/hivedemos/studentinfo.csv' INTO TABLE STUDENT_INFO;
Loading data to table students.student_info
Table students.student_info stats: [numFiles=1, totalSize=109]
OK
Time taken: 0.397 seconds
hive>
```

- sub -> is an **Array<STRING>**, where each value of it is terminated with ':'
- marks -> is represented using **MAP<STRING,INT>**, in this, STRING and INT (i.e key and value pairs) are separated by '!', and again each map value is separated by ':'

Querying a table

Objective: To retrieve the student details from “EXT_STUDENT” table.

Act:

```
SELECT * from EXT_STUDENT;
```

Outcome:

```
hive> select * from EXT_STUDENT;
OK
1001  John    3.0
1002  Jack     4.0
1003  Smith    4.5
1004  Scott    4.2
1005  Joshi    3.5
1006  Alex     4.5
1007  David    4.2
1008  James    4.0
1009  John     3.0
1010  Joshi    3.5
Time taken: 0.054 seconds, Fetched: 10 row(s)
hive>
```

Objective: Querying Collection Data Types.

Act:

```
SELECT * from STUDENT_INFO;
SELECT NAME,SUB FROM STUDENT_INFO;
// To retrieve value of Mark1
SELECT NAME, MARKS['Mark1'] from STUDENT_INFO;
// To retrieve subordinate (array) value
SELECT NAME,SUB[0] FROM STUDENT_INFO;
```

Outcome:

```
hive> SELECT * from STUDENT_INFO;
OK
1001  John    ["Smith","Jones"]    {"Mark1":45,"Mark2":46,"Mark3":43}
1002  Jack    ["Smith","Jones"]    {"Mark1":46,"Mark2":47,"Mark3":42}
Time taken: 0.044 seconds, Fetched: 2 row(s)
```

```
hive> SELECT NAME,SUB FROM STUDENT_INFO;
OK
John    ["Smith","Jones"]
Jack    ["Smith","Jones"]
Time taken: 0.061 seconds, Fetched: 2 row(s)
hive>
```

```
hive> SELECT NAME, MARKS['Mark1'] from STUDENT_INFO;
OK
John    45
Jack    46
Time taken: 0.06 seconds, Fetched: 2 row(s)
hive>
```

```
hive> SELECT NAME,SUB[0] FROM STUDENT_INFO;
OK
John    Smith
Jack    Smith
Time taken: 0.071 seconds, Fetched: 2 row(s)
hive>
```

9.5.8 Views

In Hive, view support is available only in version starting from 0.6. Views are purely logical object.

Objective: To create a view table named "STUDENT_VIEW".

Act:

```
CREATE VIEW STUDENT_VIEW AS SELECT rollno, name FROM EXT_STUDENT;
```

Outcome:

```
hive> CREATE VIEW STUDENT_VIEW AS SELECT rollno, name FROM EXT_STUDENT;
OK
Time taken: 0.606 seconds
hive>
```

Objective: Querying the view "STUDENT_VIEW".

Act:

```
SELECT * FROM STUDENT_VIEW LIMIT 4;
```

Outcome:

```
hive> SELECT * FROM STUDENT_VIEW LIMIT 4;
OK
1001    John
1002    Jack
1003    Smith
1004    Scott
Time taken: 0.279 seconds, Fetched: 4 row(s)
hive>
```

Objective: To drop the view "STUDENT_VIEW".

Act:

```
DROP VIEW STUDENT_VIEW;
```

Outcome:

```
hive> DROP VIEW STUDENT_VIEW;
OK
Time taken: 0.452 seconds
hive>
```

9.5.11 Aggregation

Hive supports aggregation functions like avg, count, etc.

Objective: To write the average and count aggregation functions.

Act:

```
SELECT avg(gpa) FROM STUDENT;
SELECT count(*) FROM STUDENT;
```

Outcome:

```
hive> SELECT avg(gpa) FROM STUDENT;
OK
3.8399999961853027
Time taken: 28.639 seconds, Fetched: 1 row(s)
hive>
```

```
hive> SELECT count(*) FROM STUDENT;
```

```
OK
10
Time taken: 26.218 seconds, Fetched: 1 row(s)
hive>
```

9.6 RCFILE IMPLEMENTATION

RCFile (Record Columnar File) is a data placement structure that determines how to store relational tables on computer clusters.

Objective: To work with RCFILE Format.

```
CREATE TABLE STUDENT_RC( rollno int, name string,gpa float ) STORED AS RCFILE;  
INSERT OVERWRITE table STUDENT_RC SELECT * FROM STUDENT;  
SELECT SUM(gpa) FROM STUDENT_RC;
```

Outcome:

```
hive> CREATE TABLE STUDENT_RC( rollno int, name string,gpa float ) STORED AS RCFILE;  
OK  
Time taken: 0.093 seconds  
hive>
```

```
hive> INSERT OVERWRITE table STUDENT_RC SELECT * from STUDENT;
```

```
hive> SELECT SUM(gpa) from STUDENT_RC;  
  
OK  
38.399999961853027  
Time taken: 25.41 seconds, Fetched: 1 row(s)  
hive>
```

Note: Stores the data in column oriented manner.

File: /user/hive/warehouse/students.db/student_rc/000000_0

Partitions

date	state	product	amount
2025-01-01	Karnataka	Shoes	500
2025-01-02	TamilNadu	Shirt	300
2025-01-03	Karnataka	Watch	800
2025-01-04	Kerala	Belt	200

Like this, Assume there are 1 crore rows of data are present

 `SELECT * FROM sales WHERE state = 'Karnataka';` If this query is executed, then it takes more time to search.

Table partition will happen based on “state”

`/user/hive/warehouse/sales/state=Karnataka/
/user/hive/warehouse/sales/state=TamilNadu/
/user/hive/warehouse/sales/state=Kerala/`

 `SELECT * FROM sales WHERE state = 'Karnataka';`

Hive will only read the Karnataka folder — making it much faster.

Each folder contains only the data for that **specific state**.

What is Partitioning?

- Partitioning in Hive means dividing a large table into smaller, more manageable pieces (called partitions) based on the values of a specific column. This makes data querying faster and more organized.
- Hive supports two types of partitioning:

1. Static Partitioning

1. The **user manually specifies the partition** (like a folder) **where data should go**. It is upon the user to mention the partition where the data from the file is to be loaded.
2. Example: If you're partitioning by STATE, you must say, "Load this data into the partition for STATE = 'A'."

2. Dynamic Partitioning

1. The **user just mentions the column** (e.g., STATE) **to be used for partitioning**.
2. Hive will **automatically create separate partitions for each unique value** in that **column** and **place the data accordingly**.

Important Points:

- In **static partitioning**, the user handles everything—from defining the partition column to loading the data into the right folder.
- If you put data for STATE = 'B' into the folder for STATE = 'A' by mistake, a query looking for STATE = 'B' will return **no results**. This is because Hive only looks at the folder (partition name), not the actual data inside it.
- In **dynamic partitioning**, Hive automatically creates folders based on the unique values in the partition column and puts the data in the right place. No manual effort is needed.

Important Settings

- By default, Hive allows **dynamic partitioning**, but it expects at least **one level of static partitioning** (this is called *strict mode*).
- If you want **fully dynamic partitioning** (no static partition at all), run this command in Hive:

```
hive> set hive.exec.dynamic.partition.mode=nonstrict;
```

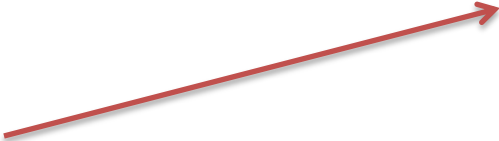
1) Static partition: It comprises columns whose values are known at compile time.

Objective:

To create a Hive table with **static partitioning** based on the gpa column.

```
CREATE TABLE IF NOT EXISTS STATIC_PART_STUDENT (  
    rollno INT,  
    name STRING  
)  
PARTITIONED BY (gpa FLOAT)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY '\t';
```

This tells the hive to create a partition based on the gpa column.



Outcome:

```
hive> CREATE TABLE IF NOT EXISTS STATIC_PART_STUDENT(rollno INT,name STRING) PARTITIONED BY (gpa FLOAT) ROW  
FORMAT DELIMITED FIELDS TERMINATED BY '\t';  
OK  
Time taken: 0.105 seconds  
hive> █
```

Objective: Load data into partition table from table.

Act:

```
INSERT OVERWRITE TABLE STATIC_PART_STUDENT PARTITION (gpa =4.0)
SELECT rollno, name from EXT_STUDENT where gpa=4.0;
```

- This means you want to transfer data from one existing table (EXT_STUDENT) into a partitioned table (STATIC_PART_STUDENT) based on the partition key (in this case, gpa).
- **INSERT OVERWRITE TABLE** : This replaces the existing data in the partition with the new data.
- **STATIC_PART_STUDENT** : The target table that is partitioned by **gpa** .
- **PARTITION (gpa=4.0)** : This is a **static partition**, meaning the partition value is fixed (here it is **4.0**).
- **SELECT rollno, name FROM EXT_STUDENT WHERE gpa=4.0;** : Data is selected from the external table where GPA is 4.0. Only **rollno** and **name** are selected since **gpa** is already specified as a static partition.

HDFS Directory:

```
/user/hive/warehouse/students.db/static_part_student/gpa=4.0
```

Objective: To add one more static partition based on “gpa” column using the “alter” statement.

Act:

```
ALTER TABLE STATIC_PART_STUDENT ADD PARTITION (gpa=3.5);  
INSERT OVERWRITE TABLE STATIC_PART_STUDENT PARTITION (gpa =4.0) SELECT  
rollno,name from EXT_STUDENT where gpa=4.0;
```

Outcome:

```
hive> ALTER TABLE STATIC_PART_STUDENT ADD PARTITION (gpa=3.5);  
OK  
Time taken: 0.166 seconds  
hive>
```

```
ALTER TABLE STATIC_PART_STUDENT ADD PARTITION (gpa=3.5);
```

- This command adds a new partition for GPA = 3.5 to the table.
- It prepares Hive to store data in a new folder specifically for that GPA value.

```
INSERT OVERWRITE TABLE STATIC_PART_STUDENT PARTITION (gpa=4.0)  
SELECT rollno, name FROM EXT_STUDENT WHERE gpa=4.0;
```

- This command loads data into the **gpa = 4.0** partition, not the new one.
- The example is repeated to show that you can load data into existing partitions after adding new ones.

```
/user/hive/warehouse/students.db/static_part_student/gpa=3.5
```

2) Dynamic Partition

Dynamic Partition have columns whose values are known only at execution time.

rollno	name	gpa
101	John	4.0
102	Alice	3.5
103	Bob	4.0
104	David	3.5

Table name: EXT_STUDENT

```
CREATE TABLE DYN_PART_STUDENT (  
    rollno INT,  
    name STRING  
) PARTITIONED BY (gpa FLOAT);  
Create the destination table with partitioning
```

```
SET hive.exec.dynamic.partition = true;  
SET hive.exec.dynamic.partition.mode = nonstrict;  
Enable dynamic partitioning in Hive settings
```

```
INSERT OVERWRITE TABLE DYN_PART_STUDENT PARTITION (gpa)  
SELECT rollno, name, gpa FROM EXT_STUDENT;
```

Insert data dynamically

```
SET hive.exec.dynamic.partition = true;
```

```
SET hive.exec.dynamic.partition.mode = nonstrict;
```

Note: The dynamic partition strict mode requires at least one static partition column. To turn this off, set hive.exec.dynamic.partition.mode=nonstrict

Bucketing

Bucketing in Hive is a way to divide data into fixed number of **files** (called **buckets**) based on the **hash of a column**.

- **Partitioning** creates directories based on column values.
- **Bucketing** creates **files** inside those directories (or even without partitioning), which improves query performance.

Why Use Bucketing?

- If a column has too many unique values (e.g., student IDs or grades), partitioning will create too many directories. Instead, we use bucketing to control the number of files — like limiting the number of "splits."

1. Create Table (STUDENT) & Load Data

```
CREATE TABLE IF NOT EXISTS STUDENT (  
    rollno INT, name STRING, grade FLOAT  
)  
  
ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t';  
  
LOAD DATA LOCAL INPATH '/root/hivedemos/student.tsv' INTO TABLE STUDENT;
```

2. Enable Bucketing

```
set hive.enforce.bucketing=true;
```

This command must be run to **enable bucketing** during insert.

3. Create Bucketed Table (STUDENT_BUCKET)

```
CREATE TABLE IF NOT EXISTS STUDENT_BUCKET (  
    rollno INT, name STRING, grade FLOAT  
)  
CLUSTERED BY (grade) INTO 3 BUCKETS;
```

4. Insert Data into Bucketed Table

```
FROM STUDENT  
INSERT OVERWRITE TABLE STUDENT_BUCKET  
SELECT rollno, name, grade;
```

Data is inserted from **STUDENT** into **STUDENT_BUCKET** and **split into 3 buckets** using hash of **grade**.

```
000000_0  
000001_0  
000002_0
```

These are 3 bucket files created by Hive

5. Accessing Specific Bucket

```
SELECT DISTINCT GRADE FROM STUDENT_BUCKET  
TABLESAMPLE(BUCKET 1 OUT OF 3 ON GRADE);
```

This returns data only from **bucket 1 of 3**, based on grade.

JOIN operation in Hive

- **JOIN operation in Hive**, similar to SQL joins. It shows how two tables (STUDENT and DEPARTMENT) are joined using a common column (rollno) as the join key.

Objective:

- Perform a **JOIN** between STUDENT and DEPARTMENT tables using rollno as the key to fetch student details along with department numbers.

Step 1: Create **STUDENT** Table

```
CREATE TABLE IF NOT EXISTS STUDENT (  
    rollno INT,  
    name STRING,  
    gpa FLOAT  
)  
ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t';
```

Step 2: Load Data into **STUDENT**

```
LOAD DATA LOCAL INPATH '/root/hivedemos/student.tsv' OVERWRITE INTO TABLE STUDENT;
```

Step 3: Create DEPARTMENT Table

```
CREATE TABLE IF NOT EXISTS DEPARTMENT (  
    rollno INT,  
    deptno INT,  
    name STRING  
)  
  
ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t';
```

Step 4: Load Data into DEPARTMENT Table

```
LOAD DATA LOCAL INPATH '/root/hivedemos/department.tsv' OVERWRITE INTO TABLE DEPARTMENT;
```

Step 5: Perform Join

```
SELECT a.rollno, a.name, a.gpa, b.deptno  
FROM STUDENT a  
JOIN DEPARTMENT b  
ON a.rollno = b.rollno;
```

- Aliases:
 - `a` for STUDENT
 - `b` for DEPARTMENT

9.8 USER-DEFINED FUNCTION (UDF)

In Hive, you can use custom functions by defining the User-Defined Function (UDF).

Objective: Write a Hive function to convert the values of a field to uppercase.

Act:

```
package com.example.hive.udf;

import org.apache.hadoop.hive.ql.exec.Description;
import org.apache.hadoop.hive.ql.exec.UDF;

@Description(
    name="SimpleUDFExample")

public final class MyUpperCase extends UDF {
    public String evaluate(final String word) {
        return word.toUpperCase();
    }
}
```

Outcome:

```
hive> ADD JAR /root/hivedemos/UpperCase.jar;
Added [/root/hivedemos/UpperCase.jar] to class path
Added resources: [/root/hivedemos/UpperCase.jar]
hive> CREATE TEMPORARY FUNCTION touppercase AS 'com.example.hive.udf.MyUpperCase';
OK
Time taken: 0.014 seconds
hive>
```

```
hive> Select touppercase (name) from STUDENT;
OK
JOHN
JACK
SMITH
SCOTT
JOSHI
ALEX
DAVID
JAMES
JOHN
JOSHI
Time taken: 0.061 seconds, Fetched: 10 row(s)
hive>
```

Note: Convert this Java Program into Jar.

```
ADD JAR /root/hivedemos/UpperCase.jar;
CREATE TEMPORARY FUNCTION touppercase AS 'com.example.hive.udf.MyUpperCase';
SELECT TOUPPERCASE(name) FROM STUDENT;
```

Chapter 2

Pig

Introduction to Pig: What is Pig, Anatomy of Pig, Pig on Hadoop, Pig Philosophy, Use case for Pig, Pig Latin Overview, Data types in Pig, Running Pig, Execution Modes of Pig, HDFS Commands, Relational Operators, Eval Function, Complex Data Types, Piggy Bank, User Defined Function, Pig Vs Hive.

What is pig?

Apache Pig is a platform for data analysis. It is an alternative to MapReduce Programming. Pig was developed as a research project at Yahoo.

Key Features of Pig

- **Engine to Run Data Steps**

- Pig has a **system (engine)** that runs steps to process data.
- It works on top of **Hadoop** and runs tasks in **parallel** across many computers.
- This makes it fast and good for handling **big data**.

- **Pig Latin – A Simple Language**

- Pig uses its own language called **Pig Latin**.
- This language is used to **write instructions** about how the data should be processed.
- It is easier to use than writing complicated programs.

- **Supports Common Data Tasks**

- In Pig Latin, you can easily do tasks like:

- **JOIN** (combine data),
- **FILTER** (remove unwanted data),
- **SORT** (arrange data), etc.

- These are similar to what you do in **SQL**.

- **User Defined Functions (UDFs)**

- If you want to do something special that Pig doesn't already have,
 - You can write your **own function**.
- These are called **UDFs** (User Defined Functions).
- You can write them in **Java, Python, etc.**

Anatomy of Pig

The main components of Pig are as follows:

1. Data flow language (**Pig Latin**).
2. Interactive shell where you can type Pig Latin statements (**Grunt**).
3. Pig interpreter and execution engine.

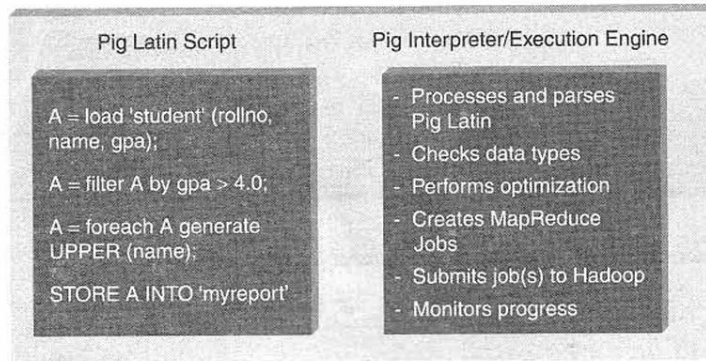


Figure 10.1 The anatomy of Pig.

Pig on Hadoop

- Pig runs on Hadoop.
 - It uses:
 - HDFS (Hadoop Distributed File System) – for storing data.
 - MapReduce – for processing the data.
- By default, Pig:
 - Reads input data from HDFS.
 - Processes the data using MapReduce.
 - Saves output back into HDFS.
 - (But Pig can also work with other data sources if needed.)

The Pig supports the following:

1. HDFS Commands

- You can access and manage files stored in HDFS.

2. UNIX Shell Commands

- Pig allows running UNIX/Linux commands directly (e.g., `ls`, `cat`).

3. Relational Operators

- Operators like `JOIN`, `FILTER`, `GROUP`, `FOREACH` – similar to SQL.

4. Positional Parameters

- Use placeholders like `$1`, `$2` in scripts to make them dynamic.

5. Common Mathematical Functions

- Functions like `SUM`, `AVG`, `MIN`, `MAX`, etc., are available.

6. Custom Functions (UDFs)

- You can write your own functions if built-in ones aren't enough.

7. Complex Data Structures

- Pig can handle advanced types like:
 - **Tuples** (like a row),
 - **Bags** (collection of tuples),
 - **Maps** (key-value pairs).

Pig Philosophy

1. **Pigs Eat Anything:** Pig can process different kinds of data such as structured and unstructured data.
2. **Pigs Live Anywhere:** Pig not only processes files in HDFS, it also processes files in other sources such as files in the local file system.
3. **Pigs are Domestic Animals:** Pig allows you to develop user-defined functions and the same can be included in the script for complex operations.
4. **Pigs Fly:** Pig processes data quickly.

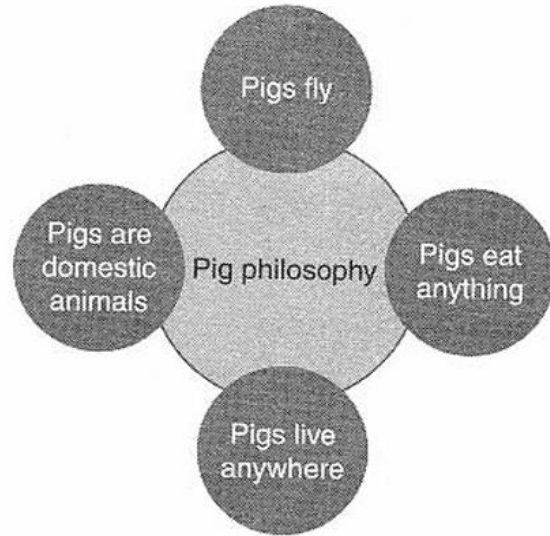


Figure 10.2 Pig philosophy.

Use Case for PIG: ETL Processing

Pig is widely used for “ETL” (Extract, Transform, and Load). Pig can extract data from different sources such as ERP, Accounting, Flat Files, etc. Pig then makes use of various operators to perform transformation on the data and subsequently loads it into the data warehouse. Refer Figure 10.3.

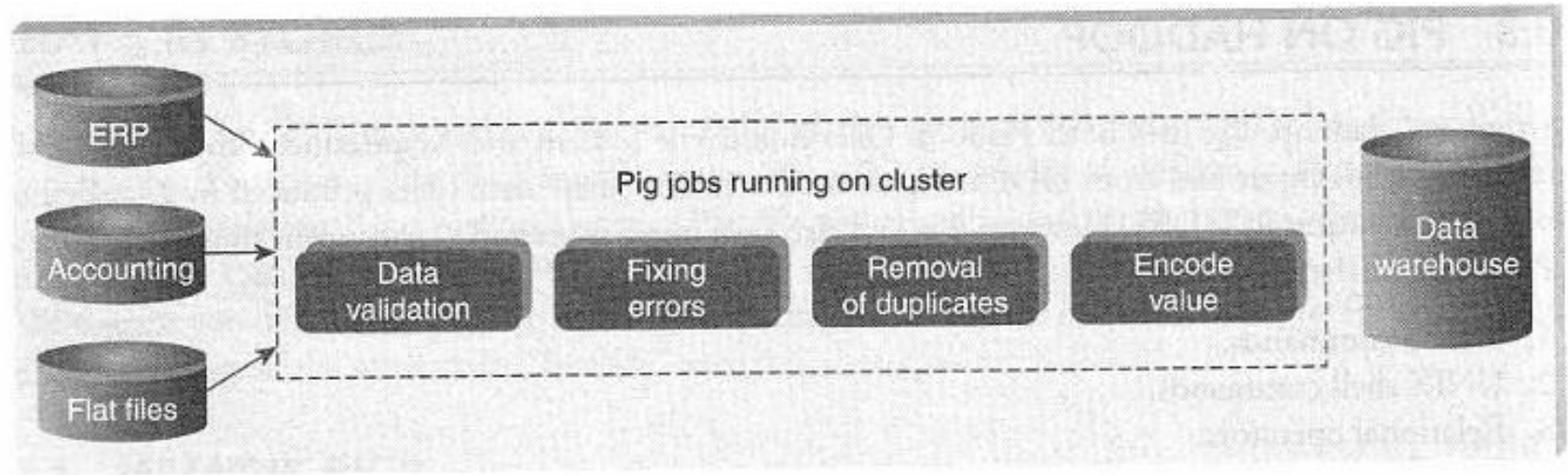


Figure 10.3 Pig: ETL Processing.

Pig Latin Overview

1. Pig Latin statements are basic constructs to process data using Pig.
2. Pig Latin statement is an operator.
3. An operator in Pig Latin takes a relation as input and yields another relation as output.
4. Pig Latin statements include schemas and expressions to process data.
5. Pig Latin statements should end with a semi-colon.

Pig Latin Statements are generally ordered as follows:

1. **LOAD** statement that reads data from the file system.
2. Series of statements to perform transformations.
3. **DUMP** or **STORE** to display/store result.

The following is a simple Pig Latin script to load, filter, and store “student” data.

```
A = load 'student' (rollno, name, gpa);  
A = filter A by gpa > 4.0;  
A = foreach A generate UPPER (name);  
STORE A INTO 'myreport'
```

Note: In the above example **A** is a relation and NOT a variable.

10.6.2 Pig Latin: Keywords

Keywords are reserved. It cannot be used to name things.

10.6.3 Pig Latin: Identifiers

1. Identifiers are names assigned to fields or other data structures.
2. It should begin with a letter and should be followed only by letters, numbers, and underscores.

Table 10.1 Valid and invalid identifiers

Valid Identifier	Y	A1	A1_2014	Sample
Invalid Identifier	5	Sales\$	Sales%	_Sales

10.6.4 Pig Latin: Comments

In Pig Latin two types of comments are supported:

1. Single line comments that begin with "--",
2. Multiline comments that begin with "/*" and end with "*/".

Table 10.1 Valid and invalid identifiers

Valid Identifier	Y	A1	A1_2014	Sample
Invalid Identifier	5	Sales\$	Sales%	_Sales

Table 10.1 describes valid and invalid identifiers.

10.6.4 Pig Latin: Comments

In Pig Latin two types of comments are supported:

1. Single line comments that begin with "--".
2. Multiline comments that begin with "/*" and end with "*/".

10.6.5 Pig Latin: Case Sensitivity

1. Keywords are *not* case sensitive such as LOAD, STORE, GROUP, FOREACH, DUMP, etc.
2. Relations and paths are case-sensitive.
3. Function names are case sensitive such as PigStorage, COUNT.

10.6.6 Operators in Pig Latin

Table 10.2 describes operators in Pig Latin.

Table 10.2 Operators in Pig Latin

Arithmetic	Comparison	Null	Boolean
+	==	IS NULL	AND
-	!=	IS NOT NULL	OR
*	<		NOT
/	>		
%	<=		
	>=		

10.7 DATA TYPES IN PIG

10.7.1 Simple Data Types

Table 10.3 describes simple data types supported in Pig. In Pig, fields of unspecified types are considered as an array of bytes which is known as bytearray.

Null: In Pig Latin, NULL denotes a value that is unknown or is non-existent.

10.7.2 Complex Data Types

Table 10.4 describes complex data types in Pig.

Table 10.3 Simple data types supported in Pig

Name	Description
Int	Whole numbers
Long	Large whole numbers
Float	Decimals
Double	Very precise decimals
Chararray	Text strings
Bytearray	Raw bytes
Datetime	Datetime
Boolean	true or false

Table 10.4 Complex data types in Pig

Name	Description
Tuple	An ordered set of fields. Example: (2,3)
Bag	A collection of tuples. Example: {(2,3),(7,5)}
map	key, value pair (open # Apache)

10.8 RUNNING PIG

You can run Pig in two ways:

1. Interactive Mode.
2. Batch Mode.

10.8.1 Interactive Mode

You can run Pig in interactive mode by invoking **grunt** shell. Type **pig** to get grunt shell as shown below.

```

root@volgalnx010:~# pig
2015-02-23 21:07:38,916 [main] INFO org.apache.pig.Main - Apache Pig version 0.12.0-cdh5.1.3 (reexported) compiled Sep 16 2014, 20:39:43
2015-02-23 21:07:38,917 [main] INFO org.apache.pig.Main - Logging error messages to: /root/pig_1424705858915.log
2015-02-23 21:07:38,934 [main] INFO org.apache.pig.impl.util.Utils - Default bootup file /root/.pigbootup not found
2015-02-23 21:07:39,313 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapred.job.tracker is deprecated. Instead, use mapreduce.jobtracker.address
2015-02-23 21:07:39,313 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - fs.default.name is deprecated. Instead, use fs.defaultFS
2015-02-23 21:07:39,313 [main] INFO org.apache.pig.backend.hadoop.executionengine.HExecutionEngine - Connecting to hadoop file system at: hdfs://volgalnx010.ad.infosys.com:9000
2015-02-23 21:07:39,800 [main] WARN org.apache.hadoop.util.NativeCodeLoader - Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
2015-02-23 21:07:40,234 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - fs.default.name is deprecated. Instead, use fs.defaultFS
grunt>

```

Once you get the grunt prompt, you can type the Pig Latin statement as shown below.

```
grunt> A = load '/pigdemo/student.tsv' as (rollno, name, gpa);  
grunt> DUMP A;
```

Here, the path refers to HDFS path and DUMP displays the result on the console as shown below.

```
(1001,John,3.0)  
(1002,Jack,4.0)  
(1003,Smith,4.5)  
(1004,Scott,4.2)  
(1005,Joshi,3.5)  
grunt>
```

10.8.2 Batch Mode

You need to create “**Pig Script**” to run pig in batch mode. Write Pig Latin statements in a file and save it with **.pig** extension.

10.9 EXECUTION MODES OF PIG

You can execute pig in two modes:

1. Local Mode.
2. MapReduce Mode.

10.9.1 Local Mode

To run pig in local mode, you need to have your files in the local file system.

Syntax:

```
pig -x local filename
```

10.9.2 MapReduce Mode

To run pig in MapReduce mode, you need to have access to a Hadoop Cluster to read/write file. This is the default mode of Pig.

Syntax:

```
pig filename
```

10.10 HDFS COMMANDS

You can work with all HDFS commands in Grunt shell. For example, you can create a directory as shown below.

```
grunt> fs -mkdir /piglatindemos;  
grunt>
```

The sections have been designed as follows:

Objective: What is it that we are trying to achieve here?

Input: What is the input that has been given to us to act upon?

Act: The actual statement/command to accomplish the task at hand.

Outcome: The result/output as a consequence of executing the statement.

10.11 RELATIONAL OPERATORS

10.11.1 FILTER

FILTER operator is used to select tuples from a relation based on specified conditions.

Objective: Find the tuples of those student where the GPA is greater than 4.0.

Input:

Student (rollno:int,name:chararray,gpa:float)

Act:

```
A = load '/pigdemo/student.tsv' as (rollno:int, name:chararray, gpa:float);
```

```
B = filter A by gpa > 4.0;
```

```
DUMP B;
```

Output:

```
(1003,Smith,4.5)
```

```
(1004,Scott,4.2)
```

```
[root@volga1nx010 pigdemos]#
```

10.11.2 FOREACH

Use **FOREACH** when you want to do data transformation based on columns of data.

Objective: Display the name of all students in uppercase.

Input:

Student (rollno:int,name:chararray,gpa:float)

Act:

```
A = load '/pigdemo/student.tsv' as (rollno:int, name:chararray, gpa:float);
```

```
B = foreach A generate UPPER (name);
```

```
DUMP B;
```

Output:

```
(JOHN)
```

```
(JACK)
```

```
(SMITH)
```

```
(SCOTT)
```

```
(JOSHI)
```

```
[root@volga1nx010 pigdemos]#
```

10.11.3 GROUP

GROUP operator is used to group data.

Objective: Group tuples of students based on their GPA.

Input:

Student (rollno:int,name:chararray,gpa:float)

Act:

```
A = load '/pigdemo/student.tsv' as (rollno:int, name:chararray, gpa:float);
```

```
B = GROUP A BY gpa;
```

```
DUMP B;
```

Output:

```
(3.0,{(1001,John,3.0),(1001,John,3.0)})
(3.5,{(1005,Joshi,3.5),(1005,Joshi,3.5)})
(4.0,{(1008,James,4.0),(1002,Jack,4.0)})
(4.2,{(1007,David,4.2),(1004,Scott,4.2)})
(4.5,{(1006,Alex,4.5),(1003,Smith,4.5)})
[root@volga1nx010 pigdemos]#
```

10.11.4 DISTINCT

DISTINCT operator is used to remove duplicate tuples. In Pig, *DISTINCT* operator works on the entire tuple and NOT on individual fields.

Objective: To remove duplicate tuples of students.

Input:

Student (rollno:int,name:chararray,gpa:float)

Input:

1001	John	3.0
1002	Jack	4.0
1003	Smith	4.5
1004	Scott	4.2
1005	Joshi	3.5
1006	Alex	4.5
1007	David	4.2
1008	James	4.0
1001	John	3.0
1005	Joshi	3.5

Act:

```
A = load '/pigdemo/student.tsv' as (rollno:int, name:chararray, gpa:float);  
B = DISTINCT A;  
DUMP B;
```

Output:

```
(1001,John,3.0)  
(1002,Jack,4.0)  
(1003,Smith,4.5)  
(1004,Scott,4.2)  
(1005,Joshi,3.5)  
(1006,Alex,4.5)  
(1007,David,4.2)  
(1008,James,4.0)  
[root@volga1nx010 pigdemos]#
```

10.11.5 LIMIT

LIMIT operator is used to limit the number of output tuples.

Objective: Display the first 3 tuples from the “student” relation.

Input:

Student (rollno:int,name:chararray,gpa:float)

Act:

```
A = load '/pigdemo/student.tsv' as (rollno:int, name:chararray, gpa:float);  
B = LIMIT A 3;  
DUMP B;
```

Output:

```
(1001,John,3.0)  
(1002,Jack,4.0)  
(1003,Smith,4.5)  
[root@volga1nx010 pigdemos]#
```

10.11.6 ORDER BY

ORDER BY is used to sort a relation based on specific value.

Objective: Display the names of the students in Ascending Order.

Input:

Student (rollno:int,name:chararray,gpa:float)

Act:

```
A = load '/pigdemo/student.tsv' as (rollno:int, name:chararray, gpa:float);
B = ORDER A BY name;
DUMP B;
```

Output:

```
(1006,Alex,4.5)
(1007,David,4.2)
(1002,Jack,4.0)
(1008,James,4.0)
(1001,John,3.0)
(1001,John,3.0)
(1005,Joshi,3.5)
(1005,Joshi,3.5)
(1004,Scott,4.2)
(1003,Smith,4.5)
[root@volga1nx010 pigdemos]#
```

10.11.7 JOIN

It is used to join two or more relations based on values in the common field. It always performs inner Join.

Objective: To join two relations namely, “student” and “department” based on the values contained in the “rollno” column.

Input:

Student (rollno:int,name:chararray,gpa:float)

Department(rollno:int,deptno:int,deptname:chararray)

Act:

```
A = load '/pigdemo/student.tsv' as (rollno:int, name:chararray, gpa:float);
B = load '/pigdemo/department.tsv' as (rollno:int, deptno:int,deptname:chararray);
C = JOIN A BY rollno, B BY rollno;
DUMP C;
DUMP B;
```

Output:

```
(1001,John,3.0,1001,101,B.E.)
(1001,John,3.0,1001,101,B.E.)
(1002,Jack,4.0,1002,102,B.Tech)
(1003,Smith,4.5,1003,103,M.Tech)
(1004,Scott,4.2,1004,104,MCA)
(1005,Joshi,3.5,1005,105,MBA)
(1005,Joshi,3.5,1005,105,MBA)
(1006,Alex,4.5,1006,101,B.E.)
(1007,David,4.2,1007,104,MCA)
(1008,James,4.0,1008,102,B.Tech)
[root@volga1nx010 pigdemos]#
```

10.11.8 UNION

It is used to merge the contents of two relations.

Objective: To merge the contents of two relations “student” and “department”.

Input:

Student (rollno:int,name:chararray,gpa:float)

Department(rollno:int,deptno:int,deptname:chararray)

Act:

```
A = load '/pigdemo/student.tsv' as (rollno, name, gp);
B = load '/pigdemo/department.tsv' as (rollno, deptno,deptname);
C = UNION A,B;
STORE C INTO '/pigdemo/uniondemo';
DUMP B;
```

Output:

“Store” is used to save the output to a specified path. The output is stored in two files: part-m-00000 contains “student” content and part-m-00001 contains “department” content.

Name	Type	Size	Replication	Block Size	Modification Time	Permission	Owner	Group
SUCCESS	file	0 B	3	128 MB	2015-02-24 17:23	rw-r--r--	root	supergroup
part-m-00000	file	146 B	3	128 MB	2015-02-24 17:23	rw-r--r--	root	supergroup
part-m-00001	file	114 B	3	128 MB	2015-02-24 17:23	rw-r--r--	root	supergroup

File: /pigdemo/uniondemo/part-m-00000

Goto: /pigdemo/uniondemo go

[Go back to dir listing](#)

[Advanced view/download options](#)

1001	John	3.0
1002	Jack	4.0
1003	Smith	4.5
1004	Scott	4.2
1005	Joshi	3.5
1006	Alex	4.5
1007	David	4.2
1008	James	4.0
1001	John	3.0
1005	Joshi	3.5

File: /pigdemo/uniondemo/part-m-00001

Goto: /pigdemo/uniondemo go

[Go back to dir listing](#)

[Advanced view/download options](#)

1001	101	S.E.
1002	102	B.Tech
1003	103	M.Tech
1004	104	MCA
1005	105	MBA
1006	101	B.E
1007	104	MCA
1008	102	B.Tech

10.11.9 SPLIT

It is used to partition a relation into two or more relations.

Objective: To partition a relation based on the GPAs acquired by the students.

- GPA = 4.0, place it into relation X.
- GPA is < 4.0, place it into relation Y.

Input:

Student (rollno:int,name:chararray,gpa:float)

Act:

```
A = load '/pigdemo/student.tsv' as (rollno:int, name:chararray, gpa:float);
SPLIT A INTO X IF gpa==4.0, Y IF gpa<=4.0;
DUMP X;
```

Output: Relation X

```
(1002,Jack,4.0)
(1008,James,4.0)
[root@volga1nx010 pigdemos]#
```

Output: Relation Y

```
(1001,John,3.0)
(1002,Jack,4.0)
(1005,Joshi,3.5)
(1008,James,4.0)
(1001,John,3.0)
(1005,Joshi,3.5)
[root@volga1nx010 pigdemos]#
```

10.11.10 SAMPLE

It is used to select random sample of data based on the specified sample size.

Objective: To depict the use of *SAMPLE*.

Input:

Student (rollno:int,name:chararray,gpa:float)

Act:

```
A = load '/pigdemo/student.tsv' as (rollno:int, name:chararray, gpa:float);
B = SAMPLE A 0.01;
DUMP B;
```

10.12 EVAL FUNCTION

10.12.1 AVG

AVG is used to compute the average of numeric values in a single column bag.

Objective: To calculate the average marks for each student.

Input:

Student (studname:chararray,marks:int)

Act:

```
A = load '/pigdemo/student.csv' USING PigStorage(',') as (studname:chararray,marks:int);
B = GROUP A BY studname;
C = FOREACH B GENERATE A.studname, AVG(A.marks);
DUMP C;
```

Output:

```
((Jack),(Jack),(Jack),(Jack)),39.75)
((John),(John),(John),(John)),39.0)
[root@voigalnx010 pigdemos]#
```

Note: You need to use PigStorage function if you wish to manipulate files other than .tsv.

10.12.2 MAX

MAX is used to compute the maximum of numeric values in a single column bag.

Objective: To calculate the maximum marks for each student.

Input:

Student (studname:chararray,marks:int)

Act:

```
A = load '/pigdemo/student.csv' USING PigStorage(',') as (studname:chararray, marks:int);
B = GROUP A BY studname;
C = FOREACH B GENERATE A.studname, MAX(A.marks);
DUMP C;
```

Output:

```
((Jack),(Jack),(Jack),(Jack)),46)
((John),(John),(John),(John)),45)
[root@voigalnx010 pigdemos]#
```

Note: Similarly, you can try the MIN and the SUM functions as well.

10.12.3 COUNT

• *COUNT* is used to count the number of elements in a bag.

Objective: To count the number of tuples in a bag.

Input:

Student (studname:chararray,marks:int)

Act:

```
A = load '/pigdemo/student.csv' USING PigStorage(',') as (studname:chararray, marks:int);
B = GROUP A BY studname;
C = FOREACH B GENERATE A.studname,COUNT(A);
DUMP C;
```

Output:

```
{{(Jack),(Jack),(Jack),(Jack)},4)
{{(John),(John),(John),(John)},4)
[root@volgalnx010 pigdemos]#
```

Note: The default file format of Pig is .tsv file. Use PigStorage() to manipulate files other than .tsv file.

10.13 COMPLEX DATA TYPES

10.13.1 TUPLE

A *TUPLE* is an ordered collection of fields.

Objective: To use the complex data type “Tuple” to load data.

Input:

(John,12)	(Jack,13)
(James,7)	(Joseph,5)
(Smith,8)	(Scott,12)

Act:

```
A = LOAD '/root/pigdemos/studentdata.tsv' AS (t1:tuple(t1a:chararray,
t1b:int),t2:tuple(t2a:chararray,t2b:int));
B = FOREACH A GENERATE t1.t1a, t1.t1b,t2.$0,t2.$1;
DUMP B;
```

Output:

```
(John,12,Jack,13)
(James,7,Joseph,5)
(Smith,8,Scott,12)
[root@volgalnx010 pigdemos]#
```

Note: You can refer to the field using Positional Notation as shown above. The Positional Notation is denoted by \$ sign and the position starts with 0 (e.g., \$0).

10.13.2 MAP

MAP represents a key/value pair.

Objective: To depict the complex data type “map”.

Input:

```
John    [city#Bangalore]
Jack    [city#Pune]
James   [city#Chennai]
```

Act:

```
A = load '/root/pigdemos/studentcity.tsv' Using PigStorage as
(studname:chararray,m:map[chararray]);
B = foreach A generate m#'city' as CityName:chararray;
DUMP B
```

Output:

```
(Bangalore)
(Pune)
(Chennai)
[root@volgalnx010 pigdemos]#
```

10.14 PIGGY BANK

Pig user can use Piggy Bank functions in Pig Latin script and they can also share their functions in Piggy Bank.

Objective: To use Piggy Bank string UPPER function.

Input:

```
Student (rollno:int,name:chararray,gpa:float)
```

Act:

```
register '/root/pigdemos/piggybank-0.12.0.jar';
A = load '/pigdemo/student.tsv' as (rollno:int, name:chararray, gpa:float);
upper = foreach A generate
org.apache.pig.piggybank.evaluation.string.UPPER(name);
DUMP upper;
```

Output:

```
(JOHN)
(JACK)
(SMITH)
(SCOTT)
(JOSHI)
(ALEX)
(DAVID)
(JAMES)
(JOHN)
(JOSHI)
[root@volga1nx010 pigdemos]#
```

Note: You need to use the “**register**” keyword to use Piggy Bank jar function in your pig script.

10.15 USER-DEFINED FUNCTIONS (UDF)

Pig allows you to create your own function for complex analysis.

Objective: To depict user-defined function.

Java Code to convert name into uppercase:

```
package myudfs;
import java.io.IOException;
import org.apache.pig.EvalFunc;
import org.apache.pig.data.Tuple;
import org.apache.pig.impl.util.WrappedIOException;
public class UPPER extends EvalFunc<String>
{
    public String exec(Tuple input) throws IOException {
        if (input == null || input.size() == 0)
            return null;
        try{
            String str = (String)input.get(0);
            return str.toUpperCase();
        }catch(Exception e){
            throw WrappedIOException.wrap("Caught exception processing input row ", e);
        }
    }
}
```

Note: Convert above java class into jar to include this function into your code.

Input:

Student (rollno:int,name:chararray,gpa:float)

Act:

```
register /root/pigdemos/myudfs.jar;
A = load '/pigdemo/student.tsv' as (rollno:int, name:chararray, gpa:float);
B = FOREACH A GENERATE myudfs.UPPER(name);
DUMP B;
```

Output:

```
(JOHN)
(JACK)
(SMITH)
(SCOTT)
(JOSHI)
(ALEX)
(DAVID)
(JAMES)
(JOHN)
(JOSHI)
[root@volga1nx010 pigdemos]#
```

10.16 PARAMETER SUBSTITUTION

Pig allows you to pass parameters at runtime.

Objective: To depict parameter substitution.

Input:

Student (rollno:int,name:chararray,gpa:float)

Act:

```
A = load '$student' as (rollno:int, name:chararray, gpa:float);
DUMP A;
```

Execute:

```
pig -param student=/pigdemo/student.tsv parameterdemo.pig
```

Output:

```
(1001,John,3.0)
(1002,Jack,4.0)
(1003,Smith,4.5)
(1004,Scott,4.2)
(1005,Joshi,3.5)
(1006,Alex,4.5)
(1007,David,4.2)
(1008,James,4.0)
(1001,John,3.0)
(1005,Joshi,3.5)
[root@volga1nx010 pigdemos]#
```

10.17 DIAGNOSTIC OPERATOR

It returns the schema of a relation.

Objective: To depict the use of *DESCRIBE*.

Input:

Student (rollno:int,name:chararray,gpa:float)

Act:

```
A = load '/pigdemo/student.tsv' as (rollno:int, name:chararray, gpa:float);
DESCRIBE A;
```

Output:

```
A: {rollno: int, name: chararray, gpa: float}
[root@volga1nx010 pigdemos]#
```

10.18 WORD COUNT EXAMPLE USING PIG

Objective: To count the occurrence of similar words in a file.

Input:

```
Welcome to Hadoop Session
Introduction to Hadoop
Introducing Hive
Hive Session
Pig Session
```

Act:

```
lines = LOAD '/root/pigdemos/lines.txt' AS (line:chararray);
words = FOREACH lines GENERATE FLATTEN(TOKENIZE(line)) as word;
grouped = GROUP words BY word;
wordcount = FOREACH grouped GENERATE group, COUNT(words);
DUMP wordcount;
```

Output:

```
(to,2)
(Pig,1)
(Hive,2)
(Hadoop,2)
(Session,3)
(Welcome,1)
(Introducing,1)
(Introduction,1)
[root@volga1nx010 pigdemos]#
```

Note:

TOKENIZE splits the line into a field for each word.

FLATTEN will take the collection of records returned by **TOKENIZE** and produce a separate record for each one, calling the single field in the record word.

10.19 WHEN TO USE PIG?

Pig can be used in the following situations:

1. When your data loads are time sensitive.
2. When you want to process various data sources.
3. When you want to get analytical insights through sampling.

10.20 WHEN NOT TO USE PIG?

Pig should not be used in the following situations:

1. When your data is completely in the unstructured form such as video, text, and audio.
2. When there is a time constraint because Pig is slower than MapReduce jobs.

10.21 PIG AT YAHOO!

Yahoo uses Pig for two things:

1. **In Pipelines**, to fetch log data from its web servers and to perform cleansing to remove companies interval views and clicks.
2. **In Research**, script is used to test a theory. Pig provides facility to integrate Perl or Python script which can be executed on a huge dataset.

10.22 PIG versus HIVE

Features	Pig	Hive
Used By	Programmers and Researchers	Analyst
Used For	Programming	Reporting
Language	Procedural data flow language	SQL Like
Suitable For	Semi - Structured	Structured
Schema/Types	Explicit	Implicit
UDF Support	YES	YES
Join/Order/Sort	YES	YES
DFS Direct Access	YES (Implicit)	YES (Explicit)
Web Interface	YES	NO
Partitions	YES	NO
Shell	YES	YES

REMIND ME

- Apache Pig is a platform for data analysis. It is an alternative to MapReduce Programming.
- It provides an **engine** for executing **data flows** (how your data should flow). Pig processes data in parallel on the Hadoop cluster.
- It provides a language called "**Pig Latin**" to express data flows.
- The main components of Pig are as follows:
 - Data flow language (**Pig Latin**).
 - Interactive shell where you can type Pig Latin statements (**Grunt**).
 - Pig interpreter and execution engine.
- You can run Pig in two ways:
 - Interactive Mode.
 - Batch Mode.

POINT ME (BOOK)

- Programming Pig, Alan Gates, O'REILY.

CONNECT ME (INTERNET RESOURCES)

- <http://pig.apache.org/docs/r0.12.0/index.html>
- <http://www.edureka.co/blog/introduction-to-pig/>
- <http://www.edureka.co/blog/pig-vs-hive/>

TEST ME

A. Fill Me

1. Pig is a _____ language.
2. In Pig, _____ is used to specify data flow.
3. Pig provides an _____ to execute data flow.
4. _____, _____ are execution modes of Pig.
5. The interactive mode of Pig is _____.
6. _____ and _____ are case sensitive in Pig.
7. _____, _____, _____ are Complex Data Types of Pig.
8. Pig is used in _____ process.

Answers:

- | | |
|-------------------------------|-----------------------|
| 1. Scripting | 5. Grunt |
| 2. Pig Latin | 6. Fields and Aliases |
| 3. Pig Engine | 7. Bag, Tuple, Map |
| 4. Local Mode, MapReduce Mode | 8. ETL |

B. Match Me

Column A	Column B
Map	Hadoop Cluster
Bag	An Ordered Collection of Fields
Local Mode	Collection of Tuples
Tuple	Key/Value Pair
MapReduce Mode	Local File System

Answers:

Column A	Column B
Map	Key/Value Pair
Bag	Collection of Tuples
Local Mode	Local File System
Tuple	An Ordered Collection of Fields
MapReduce Mode	Hadoop Cluster

C. True or False

1. PigStorage() function is case sensitive.
2. Local Mode is the default mode of Pig.
3. DISTINCT Keyword removes duplicate fields.
4. LIMIT keyword is used to display limited number of tuples in Pig.
5. ORDER BY is used for sorting.

Answers:

- | | |
|----------|---------|
| 1. True | 4. True |
| 2. False | 5. True |
| 3. False | |

ASSIGNMENTS FOR HANDS-ON PRACTICE

ASSIGNMENT 1: SPLIT

Objective: To learn about SPLIT relational operator.

Problem Description:

Write a Pig Script to split customers for reward program based on their life time values.

Input:

Customers	Life Time Value
Jack	25000
Smith	8000
David	35000
John	15000
Scott	10000
Joshi	28000
Ajay	12000
Vinay	30000
Joseph	21000

- If Life Time Value is >1000 and <= 2000 → Silver Program.
- If Life Time Value is >20000 → Gold Program.

ASSIGNMENT 2: GROUP

Objective: To learn about GROUP relational operator.

Problem Description:

Create a data file for below schemas:

- **Order:** CustomerId, ItemId, ItemName, OrderDate, DeliveryDate
 - **Customer:** CustomerId, CustomerName, Address, City, State, Country
1. Load Order and Customer Data.
 2. Write a Pig Latin Script to determine number of items bought by each customer.

ASSIGNMENT 3: COMPLEX DATA TYPE – BAG

Objective: To learn complex data type – bag in Pig.

Problem Description:

1. Create a file which contains bag dataset as shown below.

User ID	From	To
user1001	user1001@sample.com	{(user003@sample.com),(user004@sample.com), (user006@sample.com)}
user1002	user1002@sample.com	{(user005@sample.com), (user006@sample.com)}
user1003	user1003@sample.com	{(user001@sample.com),(user005@sample.com)}

2. Write a Pig Latin statement to display the names of all users who have sent emails and also a list of all the people that they have sent the email to.
3. Store the result in a file.

For Reference

Execution modes of pig

1) Local Mode

* To run in a local mode, you need to have your files in the local file system.

`pig -x local filename`

→ Command to start Apache Pig

→ option to specify the execution mode

→ runs pig in local mode (on your system using your files)

→ name of your pig script file to be executed.

2) Mapreduce mode

* To run pig in a mapreduce mode, you need to have access to a hadoop cluster to read/write file. This is the default mode of pig.

`pig -x mapreduce filename`

`pig filename`

Relational Operators

① Filter

- * It is used to select only the records (tuples) that meet a specific condition.
- * It works like a "Where" clause in SQL.

`result = FILTER relation BY condition;`

→ Keyword to filter rows.

② FOREACH

- * It is used to apply transformations to each tuple (row) in a relation.

It helps in :-

- Selecting specific columns (projection)
- Creating new columns
- Applying expressions or functions.

`result = FOREACH relation GENERATE exp1, exp2, -----;`

Ex:- Projection

```
Stud = LOAD 'students.tsc' USING PigStorage(',') AS  
(id: int, name: chararray, marks: int);
```

```
Proj = FOREACH Stud GENERATE name, marks;
```

```
DUMP Proj;
```

Ip:- Select only name and marks columns from each tuple.

Ex Q: Creating new Column

bonus = FOREACH students GENERATE name, marks,
marks + 5 AS bonus_marks;

O/P: Add 5 to each students marks and names the
new column bonus_marks.

Note: 'AS' → It is a keyword which is used to
assign a name (alias) to a field.

'USING' → It is used in the 'LOAD' or 'STORE'
Commands to specify a function or
storage mechanism for how data
should be read from or written to a
file.

'PigStorage' → It is a default loader and storer
in pig. It tells pig how to read
or write data with a specified
delimiter (like comma (,),
tab (\t), etc)

Group: It is used to group the data in a
relation based on the values of one or more fields.
* It is often used before applying aggregation
functions like COUNT, SUM, AVG etc

syntax:-

grouped_relation = GROUP relation_name BY field_name;

④ Distinct:-

* It is used to remove duplicate tuples from a relation.

dist_relation = DISTINCT relation_name;

* DISTINCT works on the entire tuples. To get distinct values of specific fields, use FOR EACH first to project the fields.

* Sorting is not guaranteed in the output.

```
d = LOAD 'Student.tsv' USING PigStorage(',') AS  
(id: int, name: chararray, marks: int);
```

```
U = FOREACH d GENERATE name;
```

```
dis = DISTINCT U;
```

```
DUMP dis;
```

O/p:- This will output a list of unique student names.

④ Limit :-

* Limit operator is used to limit the number of output tuples.

Limited_relation = LIMIT relation_name number;

Note:- To get the top N based on some order, combine ORDER BY with LIMIT.

⑤ Order by :-

* Used to sort the data in ascending (default) order or descending order based on one or more fields.

Sorted_relation = ORDER relation_name BY field_name
[ASC/DESC];

ASC → ascending order (default)

DESC → descending order

Ex:-

Sorted = ORDER data BY marks;
→ Ascending order

Sorted = ORDER data BY marks DESC;
→ Descending order

⑥ Join :-

* Join operator is used to combine two or more relations based on a common field (key) - similar to SQL joins.

joined_relation = JOIN relation1 BY field1, relation2 BY field2;

student.tsv

id: int

name: Chararray

marks: int

department.tsv

id: int

dept: Chararray.

Joins

→ Inner join (default): Includes only the matching records

→ Outer joins (Left, right, Full):

1) JOIN A BY x LEFT OUTER, B BY y;

2) JOIN A BY x RIGHT OUTER, B BY y;

3) JOIN A BY x FULL OUTER, B BY y;

Students.tsv

1	Alice	85
2	Bob	90
3	Carol	78
4	Dave	92

department.tsv

1	CSE
2	ECE
4	MECH
5	CIVIL

Left Outer Join:- It returns all the records from the left relation and the matching records from the right relation. If no matches are found, right-side fields are set to null.

Right Outer Join:-

A right outer join returns all records from the right relation and the matching records from the left relation. If there is no match in the left relation, the result will contain nulls

for the left relations' field.

Full outer join :-

- * It returns all records from both the left and right relations. If there is a match between them, it is shown. If no match, then the unmatched sides fields are null.

⑦ Union :-

- * It merges the output of two or more relations.
- * duplicate tuples are retained (in SQL, duplicate tuples are removed).

Syntax :-

`Combined = UNION relation1, relation2, ...;`

- * All input relations should have the same schema (i.e., same number and type of fields)
- * the field names can be different but the order and data types must match.
- * If you want to remove duplicates after union, you can add **DISTINCT**:

`C = DISTINCT(UNION A, B);`

Tuple

* It is an ordered collection of fields.

(John, 12)	(Jack, 13)
(James, 7)	(Joseph, 5)
(Smith, 8)	(Scott, 12)

Studentdata.tsv.

A = LOAD '/root/pigdemo/Studentdata.tsv'

AS (t1: tuple (t1a: Chararray, t1b: int),

t2: tuple (t2a: Chararray, t2b: int));

∴ t1a → name
t1b → marks } tuple 1

t2a → name
t2b → marks } tuple 2

B = FOREACH A Generate t1.t1a, t1.t1b, t2.\$0,
t2.\$1;

* You can refer to the tuple by

1) name → t1.t1a, t1.t1b

2) by position → t2.\$0, t2.\$1

Dump B

t1.t1a → name from first tuple

t1.t1b → value from first tuple.

t2.\$0 → first field from second tuple

t2.\$1 → second field from second tuple.

Output:

(John, 12, Jack, 13)

(James, 7, Joseph, 5)

(Smith, 8, Scott, 12)

Map

* A Map in Pig:

→ represents a key-value pair structure.

→ Keys are always strings

→ Values can be of any type.

John

Jack

James

Map Field

City#Bangalore

City#Pune

City#Chennai.

Key

Value

A = load '/root/pigdemos/studentcity.tsv' using

PigStorage as (studname: chararray, m: map[chararray])

↓
It tells the Pig that second column is a map where the key is a string, and the value is also a string.

B = foreach A generate m# 'city' as CityName:
Chararray;

m# 'city' → This retrieves the value associated with key city from the map m.

Dump B;

O/p:- (Bangalore)
(Pune)
(Chennai)